

Facultad de Informática / Faculty of Computer Science ???

Trabajo de Fin de Grado / Bachelor's Thesis ???

AtrilMatic: A Web-based Smart Music Stand

Por/By???
Tim Matveev y Elena Juarros
Calificación final individual/Individual final grade: ??



UNIVERSIDAD COMPLUTENSE
MADRID

Dirigido por / Supervised by ???

Manuel Freire Morán

MADRID, 2026–2027

Acknowledgments

Viva el director

Abstract

AtrilMatic: A Web-based Music-Stand

Keywords: Digital music stand, pitch detection, score following, real-time audio processing, MusicXML, WebAssembly, Rust.

Resumen

AtrilMatic, un atril inteligente

Palabras clave: Atril digital, detección de tono, seguimiento de partituras, procesamiento de audio en tiempo real, MusicXML, WebAssembly, Rust.

Contents

Acknowledgments ii
Abstract iii
Resumen iv
Chapter 1 Introduction **2**
1.1 Motivation 2
1.2 Objectives / goals ?? 2
1.3 Planning 2
Chapter 2 Related work **3**
2.1 Introduction 3
2.2 Technological foundations 3
2.3 Existing solutions 3
2.4 Comparison with the proposed solution 5
Chapter 3 Development **6**
3.1 Design 6
3.2 Implementation 8
Chapter 4 Evaluation **9**
Chapter 5 Conclusions **10**
5.1 Conclusions 10
Capítulo 1 Introducción **11**
1.1 Motivación 11
Capítulo 2 Conclusiones **12**
2.1 Conclusiones 12
Bibliography **13**
AppendixA: Evaluation questionnaire **15**

List of Figures

2.1 Tomplay user interface. 3
2.2 Oktav user interface. 4

List of Tables

Chapter 1. Introduction

1.1. Motivation

1.2. Objectives / goals ??

1.2.1. General objective

1.2.2. Specific objectives

1.3. Planning

Chapter 2. Related work

2.1. Introduction

2.2. Technological foundations

2.3. Existing solutions

Before starting with our own app, we felt that a way of gathering... was looking into already existing solutions in the market to see what we liked or not...???

This section reviews 4 applications that propose similar solutions to what we wanted to do. The solutions analysed are: Tomplay, Oktav, Newzik and Enote.

2.3.1. Tomplay

Tomplay is a mobile and desktop app developed by Tomplay.

maybe we can add here screenshot of the UI from the original web

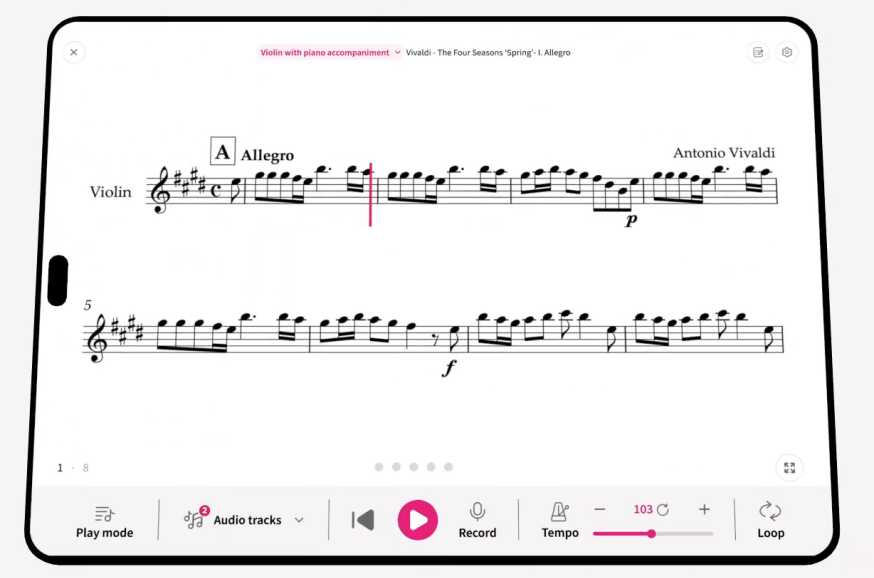


Figure 2.1: Tomplay user interface.

Its main features are:

- **Digital sheet music**: Automatically scrolls in sync with the music.
- **Note by note practice**: The score advances only when you play the right note.
- **Adjust the tempo**: Slow down or speed up the music tempo.
- **Record yourself**: Record and listen back to your performance.
- **Annotate**: Add your own annotations to the music sheet.

- **Big music sheet catalogue**: Over 100,000 scores with professional audio accompaniments.

However, it presents some limitations:

- Paid subscription
-
-

2.3.2. OKTAV

OKTAV is a web app developed by OKTAV.
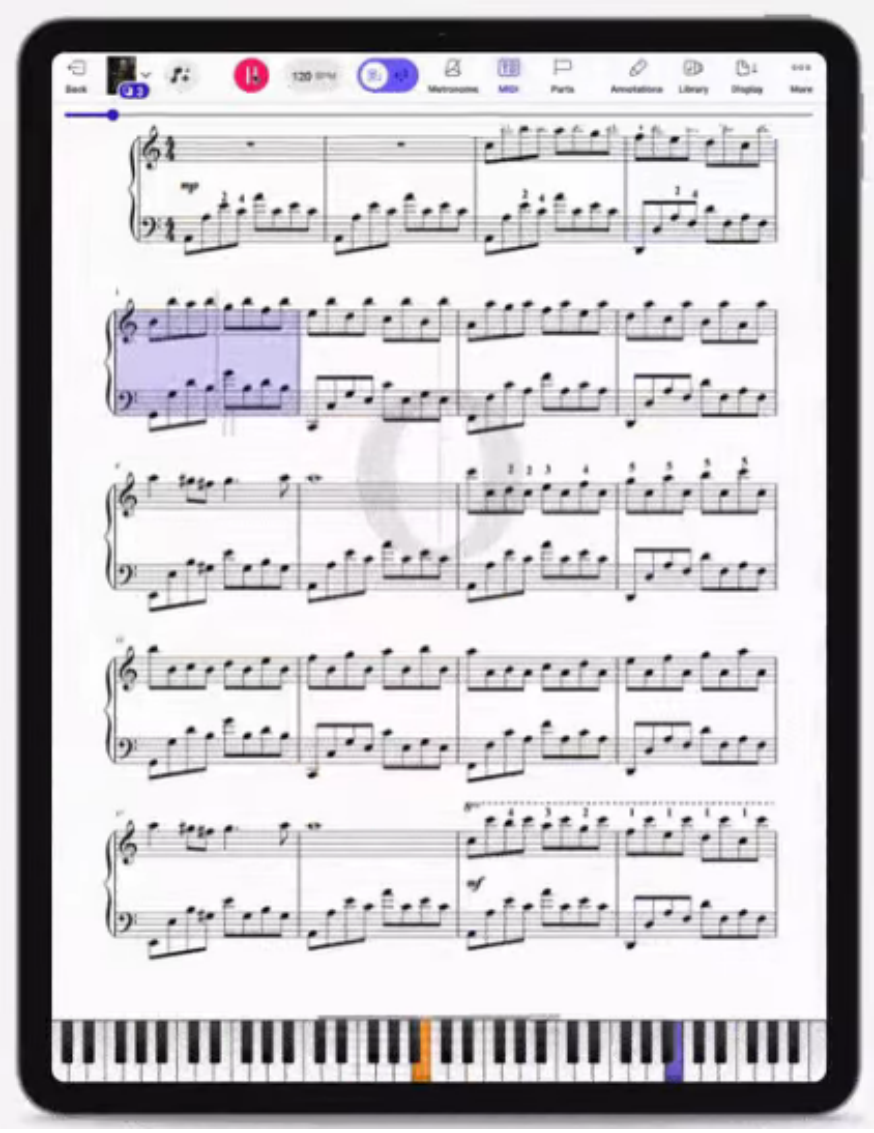


Figure 2.2: Oktav user interface.

Its main features are:

- **Autoscroll**: Play without the need to turn pages and adjust tempo.
- **Annotate**: Ability to add fingerings, notes or highlight sections.
- **Play-along**: listen and play along to sheet music based on MIDI.
- **Digital piano**: See how to play a music sheet on a visual keyboard shown along the music sheet.

However, it presents some limitations:

- Paid subscription
- Only for piano, violin and cello
-

2.4. Comparison with the proposed solution

Chapter 3. Development

3.0.1. Functional requirements

3.0.2. Non-functional requirements

3.1. Design

3.1.1. Technical Challenges

It is important to commence by outlining the actual technological needs an application like Atril would require. Recognizing pitch dynamically from microphone input could, in general, be considered a resource-intensive and in some ways non-deterministic task for a computer to perform. It is clear that a system made to perform such a task should be capable of adjusting to different microphones as well as different noise levels, and of recognizing frequencies with a minimal margin of error. The fact that, in the case of this application, such a task would have to be performed dynamically and in real time would mean that it would prove to be even more challenging. Furthermore, there is a prediction or matching element that uses the pitch data produced by the recognition system to give the best possible estimate of the position of the player in accordance with the musical composition being played. It is important to highlight that this is probably where the biggest technological focus should be allocated when it comes to an application like Atril. The recognition and prediction logic will run multiple times a second while also having to be lightweight and easy to implement into the vast world of web development frameworks, languages, and established principles.

3.1.2. WebAssembly

This is precisely where WebAssembly comes in. WebAssembly is a bytecode that acts as a compilation target for many, usually low-level, programming languages, famously C, C++, Rust, and Go. It offers simple validation, fast compilation, and interoperability with the web platform, including direct access to JavaScript and Web APIs [Haas et al., 2017].

3.1.3. Why WebAssembly?

Although WebAssembly was initially widely adopted mostly as a platform for cryptomining, this has since become a marginalized minority, comprising less than 1% of the binaries found on the web among the 8,461 WebAssembly binaries studied by the University of Stuttgart in their 2021 paper *An Empirical Study of Real-World WebAssembly Binaries* [Hilbig et al., 2021], showing a wide adoption of the format for a variety of use cases.

WebAssembly as a compilation target for the note recognition and prediction system outlined earlier is a clear choice due to several important factors. Firstly, it offers a low-overhead means of execution that will not bog down or make the rest of the proposed application heavier. Additionally, it offers a compilation target for languages that are significantly faster than JavaScript, allowing the creation of a performant library that

is still entirely interoperable with the web world and its systems via a simple API system. Finally, and importantly, WebAssembly is an abstraction over modern hardware, meaning that our application will still remain platform independent.

3.1.4. Limitations and Solutions

It is important to note the limitations of WebAssembly and the measures that can be taken to minimize their impact. Hilbig et al. [Hilbig et al., 2021], in their study of 8,461 real-world binaries, found that "vulnerabilities that propagate from insecure source languages potentially affect a wide range of binaries (e.g., two thirds of the binaries are compiled from memory unsafe languages, such as C and C++)". This matters because WebAssembly's built-in safeguards do not neutralize such bugs. The choice of source language, and its safety, is therefore crucial when developing a WebAssembly system.

3.1.5. Rust and WebAssembly

Among the languages commonly compiled to WebAssembly, Rust is the strongest candidate with respect to memory safety. Rust's compiler enforces ownership and borrowing: a value has one owner, and there can be either one mutable reference or many shared ones, never both. For safe Rust, this rules out data races and invalid memory accesses, a property that has been formally proven in machine-checked form for a realistic subset of the language and for several core standard-library abstractions [Jung et al., 2018]. Importantly, this is achieved without garbage collection, whereas languages such as Go rely on runtime memory checking and garbage collection [Xu et al., 2021]. Rust's guarantees therefore come largely from compile-time analysis rather than runtime overhead, which is essential for a real-time audio application.

3.1.6. WebAssembly Compilation Process

What follows is a small explanation of the WebAssembly compilation process, sufficient to understand the functionality of our system. In our case, Rust uses the LLVM intermediate representation, allowing a single intermediate representation to be compiled to a variety of different targets. In this case, LLVM has a WebAssembly target that converts the intermediate representation given by the Low-Level Virtual Machine into a `.wasm` module, a compact, stack-based bytecode in which high-level Rust constructs such as structs and slices are lowered to a small set of primitive numeric types and offsets into a linear memory, and in which control flow is structured [Haas et al., 2017]. Because WebAssembly can only exchange numbers with JavaScript directly, a tool such as `wasm-bindgen` post-processes the module and generates the JavaScript and TypeScript glue code that translates richer types such as strings and objects across the boundary [Rust and WebAssembly Working Group, 2026]. At load time, the browser engine validates the module against WebAssembly's type system before executing anything, which is what allows untrusted code to be run safely [Haas et al., 2017]. The format was designed to be fast to validate and compile, and the engine then translates it to machine code and runs it inside a sandbox that confines it to its own linear memory [Haas et al., 2017].

3.1.7. System architecture overview

3.1.8. Database design – we dont have DB!

3.2. Implementation

3.2.1. Backend and persistence

Chapter 4. Evaluation

4.0.1. Objectives / goals ??

4.0.2. Participants

4.0.3. Test environment

4.0.4. Evaluation tasks

4.0.5. Procedure

Chapter 5. Conclusions

5.1. Conclusions

5.1.1. Fulfilment of the objectives / goals ??

Capítulo 1. Introducción

1.1. Motivación

1.1.1. Objetivo general

1.1.2. Objetivos específicos

Capítulo 2. Conclusiones

2.1. Conclusiones

Bibliography

[Haas et al., 2017] Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A., and Bastien, J. F. (2017). Bringing the web up to speed with WebAssembly. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 185–200, Barcelona, Spain. ACM.

[Hilbig et al., 2021] Hilbig, A., Lehmann, D., and Pradel, M. (2021). An empirical study of real-world WebAssembly binaries: Security, languages, use cases. In *Proceedings of the Web Conference 2021 (WWW '21)*, pages 2696–2708, New York, NY, USA. ACM.

[Jung et al., 2018] Jung, R., Jourdan, J.-H., Krebbers, R., and Dreyer, D. (2018). RustBelt: Securing the foundations of the Rust programming language. *Proceedings of the ACM on Programming Languages*, 2(POPL):66:1–66:34.

[Rust and WebAssembly Working Group, 2026] Rust and WebAssembly Working Group (2026). `wasm-bindgen`: Facilitating high-level interactions between Wasm modules and JavaScript. https://github.com/rustwasm/wasm-bindgen. Accessed 7 November 2026.

[Xu et al., 2021] Xu, H., Chen, Z., Sun, M., Zhou, Y., and Lyu, M. R. (2021). Memory-safety challenge considered solved? an in-depth study with all Rust CVEs.

AppendixA
Evaluation questionnaire

This appendix presents ... Eg: SUS survey